

GITONE

Configurable-Shard S3-Native Git Platform

Full Solution Architecture

Design target	Value
Users	100,000,000 registered users
Git repositories	2,000,000,000 project repositories
GitOne shards	Configured once at installation (example: 256)
GitOne serving pods	One StatefulSet pod per configured shard
S3 buckets	One authoritative bucket per configured shard
Kubernetes workers	300 workers
Git transport	HTTPS / Git Smart HTTP
LFS	Git LFS supported
Implementation	Go
Persistent repository backend	S3-compatible object storage only

Core simplification

The first immutable URL component is the routing key. Every GitOne pod can accept a request, compute `xxhash64(top-level-name) % shardCount`, and stream the request directly to the owning GitOne pod. There is no separate routing service and no global repository lookup on the request path.

Contents

- 1. Executive summary
- 2. Requirements and non-goals
- 3. Architectural principles
- 4. Deterministic configurable-shard routing
- 5. Kubernetes topology
- 6. One S3 bucket per shard
- 7. Namespace and identity model
- 8. Groups, subgroups and private repositories
- 9. Control repositories and authorization
- 10. Git Smart HTTP service
- 11. S3-native Git repository format
- 12. Push and ref transactions
- 13. Clone and fetch
- 14. Git LFS
- 15. Repacking, compaction and garbage collection
- 16. Caching and local ephemeral storage
- 17. Concurrency and consistency
- 18. Capacity model
- 19. Failure handling and recovery
- 20. Security architecture
- 21. Observability and operations
- 22. Kubernetes deployment model
- 23. APIs and internal interfaces
- 24. Testing strategy
- 25. Implementation phases
- 26. Design limitations and future scale options
- 27. Final architecture summary

1. Executive summary

GitOne is a horizontally distributed Git hosting platform implemented in Go. The design deliberately reduces distributed coordination by using the immutable top-level namespace as a deterministic shard key. The platform has a shard count selected once during initial installation, represented by one GitOne StatefulSet pod and one S3 bucket per shard. A production deployment may, for example, choose 256 shards. Each shard owns all groups, subgroups, repositories, control repositories and Git LFS metadata beneath the top-level names assigned to it.

```
Request: /hello/echo/test.git/info/refs

first component = "hello"
shard = xxhash64("hello") % shardCount
      = 0..(shardCount-1)

if shard == local pod ordinal:
    serve locally
else:
    stream request to gitone-<shard>
```

All shard pods run the same binary and can receive external traffic. Kubernetes may initially deliver a request to any pod. At most one internal forwarding hop is needed. The owner pod executes authentication/authorization, namespace resolution, Git protocol handling, LFS operations, pack management and repository maintenance for its shard.

Persistent repository data is stored only in the shard bucket. Local RAM and NVMe may be used as caches and temporary workspace, but the pod can be destroyed and recreated without losing authoritative data. This keeps Kubernetes scheduling and disaster recovery simple.

Property	Decision
Routing	xxhash64(top-level namespace) % shardCount
Top-level names	Immutable
Login names	Immutable and equal to the user personal root group
Subgroup/repository names	Rename allowed beneath top level
Request hops	0 when request lands on owner, otherwise exactly 1 internal forwarding hop
Durable Git storage	Shard-specific S3 bucket
Durable local disk	Not required
Database on Git hot path	None
Separate router tier	None
Git LFS	Same shard routing; payload stored in shard bucket

2. Requirements and non-goals

2.1 Required capabilities

- 100 million registered users.
- 2 billion project Git repositories.
- A deployment-time shard-count parameter; one deterministic GitOne shard per configured slot.

- One S3 bucket per configured shard.
- Run on a Kubernetes cluster with 300 workers.
- Personal namespaces: the immutable login name is also the user personal root group.
- User-created top-level groups and unlimited logical subgroup nesting subject to configurable policy limits.
- Invitation of another user by immutable login name, with explicit acceptance.
- Private repositories by default for personal work; public/internal visibility can be added as policy.
- Git Smart HTTP clone, fetch and push using ordinary Git clients.
- Git LFS batch API, upload/download, optional locking and quota support.
- Git-backed control repositories for group ACL policy.
- S3 as the only authoritative repository and LFS data store.
- Online repacking and garbage collection without stopping normal Git traffic.

2.2 Deliberate non-goals in the base design

- No top-level namespace rename. This removes shard migration and routing exceptions.
- No separate distributed routing database.
- No NFS/CephFS/shared POSIX filesystem for Git repositories.
- No requirement for a database to map a repository URL to a shard.
- No per-repository S3 bucket.
- No cross-shard group tree; every top-level namespace and everything below it belongs to exactly one shard.
- No transparent automatic splitting of one very large top-level namespace across several shards in v1.

3. Architectural principles

Principle	Effect
Deterministic ownership	Every node can independently calculate the owner of a URL.
Immutable top-level key	Shard assignment never changes.
One shard = one bucket	Failure, quota, lifecycle and storage operations have a clear boundary.
S3 authoritative	Pods are replaceable. Local storage is cache/workspace only.
Immutable Git packs	Normal operations add new immutable data and atomically publish a new state.
Single-shard ACL evaluation	Group inheritance does not require distributed authorization calls.
Streaming forwarding	Large Git/LFS bodies are not buffered by the first pod.
No global hot-path catalog	Routing does not depend on a global metadata service.

Consistency boundary

The top-level namespace is simultaneously the URL ownership boundary, ACL inheritance root, Git shard boundary and S3 bucket selection key.

4. Deterministic configurable-shard routing

4.1 Routing algorithm

```
func shardForTopLevel(name string, shardCount uint64) uint64 {
    // Canonicalize before hashing: validated lowercase login/group key.
    if shardCount == 0 {
        panic("shardCount must be greater than zero")
    }
    return xxhash.Sum64String(name) % shardCount
}
```

The system **MUST** define one canonical encoding for the top-level name before hashing. The simplest policy is an ASCII-safe lowercase identifier syntax and immutable case. Names that differ only by case must not be allowed.

URL	Routing key	Owner
/hello/echo/test.git	hello	xxhash64("hello") % shardCount
/alice/project.git	alice	xxhash64("alice") % shardCount
/acme/platform/linux.git	acme	xxhash64("acme") % shardCount
/acme/security/scanner.git	acme	same owner as all other acme paths

4.2 Installation-time shard count

During the first GitOne installation, the administrator **MUST** choose shardCount. GitOne persists this value as cluster identity/configuration and all pods must start with the same value. The StatefulSet replica count and required S3 bucket count are derived directly from shardCount. Once any top-level namespace has been created, shardCount is immutable unless a dedicated offline/online migration procedure is explicitly performed.

```
# example initial installation
GITONE_SHARD_COUNT=256

StatefulSet replicas = GITONE_SHARD_COUNT
S3 buckets           = GITONE_SHARD_COUNT
owner(name)          = xxhash64(name) % GITONE_SHARD_COUNT
```

Parameter	Rule
shardCount	Required positive integer selected during first setup
StatefulSet replicas	Exactly shardCount owner pods in the base design
S3 buckets	Exactly shardCount; bucket N is authoritative for shard N
Routing	xxhash64(canonicalTopLevelName) % shardCount
Persistence	Store shardCount in immutable cluster configuration/manifest
Restart/restore	Must use the original shardCount
Changing shardCount	Not supported as a normal configuration change; requires data migration

4.3 Every pod is also a router

```

External Service
|
+--> gitone-021 --- hash ---> gitone-173 ---> bucket-173
|
+--> gitone-173 -----> bucket-173
|
+--> gitone-244 --- hash ---> gitone-173 ---> bucket-173

```

Maximum internal forwarding: one hop.

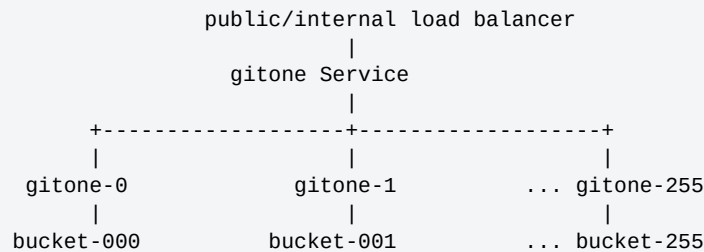
The receiving pod parses only enough of the URL to identify the top-level component. If it is not the owner, it forwards the HTTP request directly to the stable StatefulSet identity of the owner. The destination pod handles the request and the first pod streams the response back to the client.

4.3 Forwarding loop prevention

- Owner is calculated identically by every pod.
- Forwarded requests include an internal marker such as X-GitOne-Forwarded: 1 generated only by authenticated pod-to-pod communication.
- A destination that receives a forwarded request but calculates a different owner returns an internal routing error instead of forwarding again.
- Canonical URL parsing is shared code, not independently implemented handlers.

5. Kubernetes topology

Kubernetes cluster: 300 workers



Headless service provides stable pod DNS:

```

gitone-0.gitone-headless.<ns>.svc
...
gitone-255.gitone-headless.<ns>.svc

```

The configured GitOne pods are logical shards, not physical worker assignments. Kubernetes may move a pod to another worker after failure. The pod ordinal is the shard ID, so the identity remains stable even when the physical worker changes.

5.1 Worker allocation

Use	Initial design
GitOne shard pods	N logical shard pods; N is the installation shardCount
Kubernetes workers	300
Placement	Pod anti-affinity/topology spread to avoid unnecessary co-location
Remaining cluster capacity	Used for ingress/load balancer components, repack/GC jobs,

	observability, system workloads and rolling updates
Persistent volumes	Optional ephemeral/cache volumes only; no authoritative repository PV
Pod disruption	Maintain shard availability through careful rolling update and fast rescheduling; optional standby replicas can be added later

Because there are more workers than primary shard pods, it is possible to spread primary pods broadly while preserving spare scheduling capacity. Actual CPU/RAM sizing must be derived from workload benchmarks: 2 billion repositories is a storage/catalog scale target, not a statement that all repositories are simultaneously active.

6. One S3 bucket per shard

GitOne uses exactly one repository bucket per configured shard. The bucket number equals the owner shard number. For a 256-shard installation, names could run from gitone-prod-shard-000 through gitone-prod-shard-255. The shard count is chosen during initial cluster setup and then treated as immutable for that installation.

```
gitone-prod-shard-173/
|
+-- namespaces/
|   +-- hello/
|       +-- root.json
|       +-- paths/...
|
+-- repos/
|   +-- <repo-id>/
|       +-- state
|       +-- states/
|       +-- packs/
|       +-- indexes/
|       +-- txn/
|
+-- groups/
|   +-- <group-id>/...
|
+-- lfs/
|   +-- <repo-id>/sha256/...
|
+-- maintenance/
|   +-- gc/
|   +-- repack/
```

6.1 Why one bucket per shard

- The pod-to-bucket mapping is fixed and requires no lookup.
- S3 request rates, quotas, lifecycle policies and metrics are naturally isolated per shard.
- A storage incident is bounded to one shard.
- Data export, restore and audit can be performed shard by shard.
- Bucket policies can grant each shard identity access only to its own bucket.
- Operational tooling can process one shard without listing objects belonging to other shards.

6.2 S3 credentials

Prefer workload identity or short-lived credentials. GitOne pod N should receive permissions only for bucket N. A pod forwarding a request does not access the remote shard bucket; only the destination shard pod does. Repack or recovery jobs for a shard use the same restricted identity or a dedicated maintenance identity scoped to that bucket.

7. Namespace and identity model

7.1 Personal root group

Every registered user owns a personal root group whose URL name equals the immutable login name. This avoids a second concept for user-owned repositories: a user namespace is simply a special top-level group.

```
login: alice
personal root group: alice/

alice/project-a.git
alice/project-b.git
alice/lab/experiment.git
```

The login name is the shard key and therefore cannot be renamed. A separate display name and profile attributes may be changed freely without affecting URLs or routing.

7.2 Normal top-level groups

An authorized user may create a normal top-level group such as acme. The name is checked for global uniqueness and then becomes immutable. All subgroups and repositories under acme remain on the shard chosen from acme forever.

```
acme/
+-- platform/
|   +-- linux.git
|   +-- kubernetes.git
+-- applications/
|   +-- portal/
|       +-- frontend.git
|       +-- backend.git
+-- security/
    +-- scanner.git
```

7.3 Global name uniqueness

Because users and top-level groups share the same URL namespace, the system must prevent a login and a top-level group from using the same key. This is one area that requires a globally correct account/group-creation control plane. It is not on the Git request hot path. A small strongly consistent identity/namespace registry, or a globally serialized name-allocation service, can be used for creation only. After allocation, normal Git routing remains pure hash-based.

Important boundary

The no-global-database goal applies to normal Git routing and repository access. Unique allocation of new top-level names across 100 million users still needs an authoritative account/identity service. That service is not required to locate an existing repository request.

8. Groups, subgroups and private repositories

8.1 Visibility

Visibility	Meaning
private	Only explicitly authorized users/groups and inherited members may read or write. Recommended default.
internal	Any authenticated GitOne user may read; write still follows ACL. Optional feature.
public	Anonymous read allowed; write still authenticated and authorized. Optional feature.

8.2 Invitations by login name

```

Alice owns acme/
Alice invites login "bob" as Developer
    |
    v
GitOne resolves bob -> immutable user ID
    |
    v
pending invitation on acme shard
    |
+----+----+
|      |      |
| Accept  Decline
|      |      |
    v
membership becomes active

```

The human interface uses the immutable login name. The stored membership references the immutable user identifier, preventing accidental permission loss if profile data changes.

8.3 Inheritance

```

acme/                bob = developer
|
+-- platform/        inherited
|   +-- linux.git    inherited
|   +-- kube.git     inherited
|
+-- applications/    inherited
|   +-- portal.git   inherited

```

A user can also be granted access directly to a subgroup or repository. Direct grants must not accidentally grant access to sibling paths.

9. Control repositories and authorization

GitOne retains the control-repository concept. A group may contain a reserved control repository that defines membership and policy in a reviewable Git history. The control repository itself remains on the same shard because it shares the same top-level path.

```
acme/
+-- control.git
+-- platform/
|   +-- control.git
|   +-- linux.git
+-- security/
    +-- scanner.git
```

A human-readable control file can reference login names, while the shard compiles the effective ACL to immutable user IDs for runtime evaluation.

```
{
  "members": {
    "alice": "owner",
    "bob": "developer",
    "charlie": "reader"
  }
}
```

9.1 Authorization lookup

```
request /acme/platform/linux.git
      |
      v
owner shard = hash("acme")
      |
      v
load compiled ACL for:
  acme
  acme/platform
  acme/platform/linux.git
      |
      v
authorize operation
```

Because the entire tree is in one shard, ACL inheritance requires no cross-shard call. Effective ACL data should be cached in RAM and versioned against the S3-backed group/control state.

10. Git Smart HTTP service

The service exposes standard Git Smart HTTP endpoints so ordinary Git clients work without custom helpers. The principal operations are repository discovery plus upload-pack for clone/fetch and receive-pack for push.

```
GET  /<path>.git/info/refs?service=git-upload-pack
POST /<path>.git/git-upload-pack

GET  /<path>.git/info/refs?service=git-receive-pack
POST /<path>.git/git-receive-pack
```

10.1 Streaming rule

The first GitOne pod must proxy bodies and responses as streams. It must not read a complete push, clone or LFS object into memory before forwarding.

```

client
| streaming HTTP
v
gitone-42
| streaming internal HTTP
v
gitone-173
| pack/LFS processing
v
bucket-173

```

11. S3-native Git repository format

GitOne should not emulate a POSIX .git directory object by object. The durable representation should be optimized for object storage: immutable packs and immutable snapshots, plus one small mutable state/commit-point object per repository.

```

repos/<repo-id>/
|
+-- state                               mutable CAS publication point
+-- states/
|   +-- <generation>.json              immutable repository snapshots
+-- packs/
|   +-- <pack-id>.pack                 immutable
|   +-- <pack-id>.idx                 immutable
+-- indexes/
|   +-- ...                           optional derived indexes
+-- txn/
|   +-- <transaction-id>/...          temporary/uncommitted data

```

11.1 Repository state example

```

{
  "generation": 184,
  "default_branch": "refs/heads/main",
  "refs_snapshot": "states/184-refs.json",
  "pack_manifest": "states/184-packs.json"
}

```

The state object is small and changes only when publishing a new repository generation. Packfiles and snapshots are immutable. This keeps failure recovery and concurrent readers simple.

12. Push and ref transactions

A normal Git push already delivers a pack stream. GitOne validates that pack, builds/validates its index, uploads immutable data to S3 and only then attempts to publish new refs.

```

git push
|
v
receive-pack
|
+-- read repository state + version/ETag
+-- validate requested old/new refs
+-- receive pack into ephemeral workspace / stream staging
+-- validate pack and object graph
+-- run authorization and policy checks
+-- upload immutable pack/index
+-- write immutable new refs/pack snapshot
+-- conditional publish of repo state
v
success

```

12.1 Concurrent pushes

The publication step must be compare-and-swap. If two pushes start from the same repository state, only one may publish that expected old generation. The loser rereads current state and is rejected or safely retried according to Git ref semantics.

```

state 100
| | +-- push B -> candidate 101B -- CAS fails
| |
| +---- push A -> candidate 101A -- CAS succeeds

```

Old packs remain immutable throughout.

Unpublished packs are harmless unreachable objects and are later removed by maintenance GC.

13. Clone and fetch

For clone and fetch, the owner shard advertises refs, negotiates wants/haves, traverses reachable objects and returns a pack stream. The implementation should cache small pack indexes and repository manifests in RAM so object location usually does not require S3 metadata requests.

```

client wants commits
|
v
ref snapshot
|
v
object traversal / reachability
|
+--> RAM pack index cache
|      |
|      +--> S3 Range GET from immutable packs
|
v
outgoing pack stream --> client

```

For very hot repositories, an ephemeral local pack cache on NVMe can reduce repeated S3 range reads. Cache loss affects performance only, never correctness.

14. Git LFS

Git LFS uses the same URL routing rule because the repository path still begins with the top-level namespace. Authorization is performed by the owner shard. LFS content is stored in the same shard bucket as the repository to preserve isolation and simplify deletion/accounting.

```

bucket-173/
  lfs/
    <repo-id>/
      sha256/
        ab/
          <full-object-id>

```

14.1 LFS operations

- Batch endpoint determines which objects exist and which require upload/download.
- Upload verifies the expected SHA-256 object ID before publication.
- Download can be streamed through GitOne initially; later, pre-signed S3 URLs may offload large payload transfer if policy permits.
- LFS locking metadata is shard-local and stored with repository metadata.
- Quotas are evaluated per user/group/repository using shard-maintained counters or derived accounting.
- Deleting a repository schedules LFS cleanup after the configured retention/grace period.

15. Repacking, compaction and garbage collection

Repeated pushes create many packs. GitOne periodically compacts them into fewer optimized packs. Repack is a copy-on-write operation: existing packs remain readable while a new pack set is built.

```

pack-001 --+
pack-002 --+
pack-003 --+--> repack worker --> pack-new.pack + pack-new.idx
...      |
pack-500 --+

publish new generation by CAS
    |
    v
new readers use new pack set
old pack set retained for grace period
    |
    v
GC deletes unreachable old packs

```

15.1 Where repack runs

A shard pod may perform small maintenance itself, while expensive repacks should be Kubernetes Jobs created with the shard identity and bucket permission. Jobs use ephemeral local NVMe/RAM and upload only final immutable output. The repository state is changed only after successful upload.

15.2 Live compaction during push

GitOne should actively prevent repositories from accumulating hundreds or thousands of small S3 pack objects. The objective is not to impose a maximum pack size. A large client-generated pack is acceptable. The problem to control is pack fragmentation and S3 object count.

The push path evaluates the current repository pack set together with the incoming client pack. If fragmentation is below the configured thresholds, GitOne publishes the incoming pack directly. If the repository has accumulated too many small packs, GitOne may compact only those small packs together with the incoming pack before atomically publishing the new repository generation.

```

client push
    |
    v
receive + validate incoming PACK
    |

```

```

+--> fragmentation below threshold --> keep incoming pack --> publish state
|
+--> too many small packs
    |
    v
    select small packs + incoming pack
    |
    v
    live compaction
    |
    v
    fewer larger packs
    |
    v
    publish state by CAS

```

Large existing packs are not rewritten merely because a push occurs. For example, if a repository contains 8 GiB and 4 GiB packs plus sixteen small packs, GitOne should leave the two large packs untouched and compact the small packs plus the incoming push into one or a few larger packs.

- Accept client-generated packs regardless of size, subject to normal repository/object validation and operational upload limits.
- Use pack count and small-pack count as the primary compaction signals rather than a hard maximum pack size.
- Recommended initial threshold: compact when total pack count exceeds 32 or small-pack count exceeds 16.
- Recommended initial small-pack definition: less than 128 MiB. These values are configuration defaults and must be benchmarked.
- Live compaction must operate only on the selected fragmented subset; it must not trigger a full-repository repack for an ordinary push.
- If the estimated live-compaction work exceeds configured latency or resource limits, publish the incoming pack normally and enqueue background compaction instead.
- The push is acknowledged only after the selected canonical pack set is uploaded and the repository state/refs are atomically committed.
- Temporary processing may use pod-local NVMe or RAM, but S3 remains the only authoritative durable repository store.

Suggested configuration:

```

git:
  pack:
    compact_on_push: true
    max_pack_count: 32
    max_small_pack_count: 16
    small_pack_threshold: 128MiB
    live_compaction:
      max_input_bytes: 2GiB
      max_input_packs: 32

```

These limits protect push latency. They do not cap the size of a stored pack. A single 8 GiB or larger pack can remain canonical if it is otherwise valid and useful. The policy is deliberately optimized for fewer S3 objects rather than uniform pack sizes.

15.3 Safe GC

- Retain recent repository generations for a configured safety period.
- Never delete a pack referenced by any retained generation.
- Use transaction timestamps to remove abandoned uploads only after a grace period.

- Consider S3 versioning/object lock according to recovery requirements, but do not depend on them for normal repository correctness.
- Shard GC operates only inside the shard bucket.

16. Caching and local ephemeral storage

Cache	Purpose	Authoritative?
RAM namespace cache	Resolve subgroup/repository metadata	No
RAM ACL cache	Fast effective authorization	No
RAM pack-index cache	OID -> pack/offset lookup	No
NVMe pack cache	Hot pack/range caching	No
NVMe transaction workspace	Receive/repack temporary data	No
S3 shard bucket	Repository, group, LFS and durable metadata	Yes

A pod restart may rebuild caches lazily. GitOne must never require recovery of a local PV to regain repository correctness.

17. Concurrency and consistency

The design avoids distributed consensus for ordinary repository access because exactly one logical shard owns each top-level namespace. Multiple concurrent HTTP requests may still execute inside that shard, so repository mutation must use optimistic concurrency and ref preconditions.

Operation	Concurrency model
Read/clone/fetch	Concurrent; pinned to a repository generation/snapshot for request consistency
Push	Validate against expected refs and publish with conditional state update
Group/ACL update	Versioned shard-local state; control repository updates become new ACL generations
Repack	Build new immutable data, then conditional publish
GC	Delete only objects proven unreachable from all retained generations
LFS upload	Content-addressed; publish only after checksum validation

18. Capacity model

18.1 Average logical distribution

Metric	Total	Average per shard (256 reference deployment)
Registered users	100,000,000	390,625

Project repositories	2,000,000,000	7,812,500
Primary GitOne pods	N	1 owner pod per shard
S3 buckets	N	1 bucket per shard
Kubernetes workers	300	For N=256: ~0.85 primary shard pod per worker before other workloads

These are averages. xxHash provides an even distribution when namespace names are not adversarial, but routing is by top-level namespace rather than repository. Therefore a huge organization with millions of repositories remains entirely on one shard. The platform must set per-namespace service limits and monitor shard skew.

18.2 Storage examples

Average compressed Git data per project repo	2B repos total Git data	Average per shard
10 MB	20 PB	~78 TB
50 MB	100 PB	~391 TB
100 MB	200 PB	~781 TB
500 MB	1 EB	~3.9 PB

LFS is additional. Replication/erasure-coding overhead is an object-storage concern and must be included in physical capacity planning. At this scale, total bytes and request rates will normally dominate hardware design more than raw repository count.

18.3 Metadata strategy at millions of repos per shard

Do not load all repository state into memory. Use deterministic S3 keys for path metadata and keep only hot entries in RAM. Background shard indexes may be compacted into immutable index segments to accelerate listing/search, but repository access by exact path should remain direct and not require scanning the bucket.

19. Failure handling and recovery

Failure	Expected behavior
Non-owner receiving pod dies	Client connection fails; retry may land on another pod. No data loss.
Owner shard pod dies	Only that shard is temporarily unavailable until StatefulSet pod is recreated. Bucket remains authoritative.
Worker dies	Kubernetes reschedules the affected shard pod.
Repack job dies	Old generation remains valid; incomplete new objects are later GC candidates.
Push dies before state publish	New pack may be orphaned; refs unchanged.
Push dies after successful state publish	New generation is authoritative and recoverable from S3.
Bucket unavailable	That shard is unavailable; other 255 shards continue.
Entire cluster loss	Recreate the cluster with the original shard-count parameter; rebuild caches from the corresponding shard buckets and identity/config backups.

19.1 Optional future standby

If shard restart time is unacceptable, each logical shard can later have an active and standby serving replica. That changes pod count and scheduling but does not change hash routing or bucket ownership. The base design intentionally starts with one primary owner pod per shard to remain simple.

20. Security architecture

- External TLS terminates at an approved ingress/load balancer or directly in GitOne according to deployment policy.
- Pod-to-pod forwarding uses authenticated mTLS or Kubernetes workload identity so user-controlled headers cannot impersonate internal routing.
- Owner shard performs the final authorization decision. The first receiving pod is not trusted to grant repository access.
- Each shard workload identity has S3 permission only to its own bucket.
- Top-level namespace and login validation prevents path traversal, ambiguous Unicode and case-collision attacks.
- Git pack and LFS content are validated before publication.
- Control repository changes are audited as normal immutable Git history and may require protected-branch policy.
- Administrative operations and group creation are separately audited.
- Rate limits exist per source identity, user, top-level namespace and shard.

20.1 Internal request identity

A forwarded request should carry either the original credentials for revalidation or an authenticated internal identity envelope. Do not trust arbitrary X-User or X-GitOne-* headers arriving from the external interface. Strip such headers before internal forwarding.

21. Observability and operations

Metric family	Examples
Routing	requests received, local-hit %, forwarded %, routing mismatch errors
Git	clone/fetch/push count, bytes, negotiation time, pack generation time
S3	GET/PUT/range requests, bytes, latency, throttles/errors per bucket
LFS	batch calls, objects/bytes uploaded/downloaded, checksum failures
Repositories	repo count, active repos, refs, pack count distribution
Maintenance	repack queue, GC queue, orphan bytes, repack duration
Shard health	CPU, RAM, cache hit rate, active streams, queue depth
ACL	authorization latency, cache hit/miss, denied operations

All telemetry should carry shard ID, top-level namespace ID and repository ID where appropriate, but sensitive repository names or paths should be controlled by logging policy. High-cardinality metrics should use IDs carefully to avoid overwhelming the monitoring platform.

22. Kubernetes deployment model

22.1 StatefulSet

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: gitone
spec:
  serviceName: gitone-headless
  replicas: <shardCount>
  podManagementPolicy: Parallel
  template:
    spec:
      containers:
      - name: gitone
        image: gitone:<version>
        env:
        - name: POD_NAME
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
        # ordinal extracted from gitone-N
```

A standard Service selects all GitOne pods for initial request distribution. A headless Service provides stable direct addresses for internal forwarding. NetworkPolicy should permit shard-to-shard HTTP only on the internal GitOne port and restrict S3 egress according to the environment.

22.2 Scheduling

- Use topologySpreadConstraints / anti-affinity across workers and failure domains.
- Assign realistic requests/limits from measured Git stream and pack-generation workloads.
- Repack Jobs should use separate scheduling priority/resources so maintenance cannot starve interactive Git traffic.
- Use PodDisruptionBudgets carefully; with a single primary per shard, planned eviction makes that shard unavailable during restart.
- Rolling updates should use controlled partitioning or small batches and health checks before proceeding.

23. APIs and internal interfaces

23.1 Core Go interfaces

```
type ShardID uint8

type Router interface {
    Owner(topLevel string) ShardID
    Forward(ctx context.Context, shard ShardID, w http.ResponseWriter, r *http.Request)
}

type RepositoryStore interface {
    LoadState(ctx context.Context, repoID string) (RepositoryState, Version, error)
    CompareAndSwapState(ctx context.Context, repoID string, expected Version, next RepositoryState) error
    PutImmutable(ctx context.Context, key string, r io.Reader, size int64) error
    Get(ctx context.Context, key string) (io.ReadCloser, error)
    GetRange(ctx context.Context, key string, off, length int64) (io.ReadCloser, error)
}

type Authorizer interface {
    Check(ctx context.Context, subject UserID, repo RepoID, action Action) error
}
```

23.2 HTTP routing middleware

```
func shardMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        top, err := firstCanonicalPathComponent(r.URL.Path)
        if err != nil { http.Error(w, "bad path", 400); return }

        owner := uint8(xxhash.Sum64String(top))
        if owner == localShard {
            next.ServeHTTP(w, r)
            return
        }

        forwardStream(owner, w, r)
    })
}
```

24. Testing strategy

24.1 Correctness tests

- Known xxHash vectors and stable shard assignment across versions/architectures.
- Canonical path parsing, percent encoding, .git suffix handling and malicious path rejection.
- All pods compute the same destination for the same top-level key.
- No top-level rename API exists; login key is immutable.
- Concurrent pushes to the same ref cannot both publish incompatible state.
- Atomic multi-ref push behavior.
- Crash at every stage of push and repack leaves an old or new valid generation, never partial refs.
- ACL inheritance across group/subgroup/repository paths.
- Private repository access is denied without effective membership.
- Invitation acceptance and revocation.
- LFS checksum, missing object, locking and authorization behavior.
- GC never deletes packs referenced by retained generations.

24.2 Scale tests

- Synthetic 100M user namespace distribution across the configured shard count (256 shown as an example).
- Synthetic catalog representing 2B repositories without requiring 2B live Git processes.
- Shard skew tests with very large organizations.
- Large concurrent clone/fetch streams and S3 Range GET behavior.
- Concurrent pushes to hot repositories.
- LFS multi-GB upload/download streams.
- Repack under normal serving load.
- Worker/pod/bucket fault injection.

24.3 Acceptance gates

Do not claim that 300 workers satisfy a particular request rate until benchmarked with representative repository sizes, clone/fetch/push mixes, LFS traffic, S3 latency and network bandwidth. The architecture supports the namespace scale; throughput capacity is an engineering benchmark requirement.

25. Implementation phases

Phase	Deliverable
1 - Routing core	Configurable-shard xxHash function, canonical path parser, StatefulSet ordinal ownership, direct streaming proxy.
2 - Namespace/identity	Personal root groups, immutable logins/top-level groups, subgroup/repository metadata, invitations.
3 - S3 repository engine	Repository state, immutable pack storage, CAS state publishing.
4 - Git push	receive-pack, pack validation, ref transaction and authorization.
5 - Git clone/fetch	upload-pack, pack lookup/traversal and outgoing pack streaming.
6 - ACL/control.git	Group inheritance and compiled ACL cache.
7 - Git LFS	Batch/upload/download, checksum and locking.
8 - Maintenance	Repack, generation retention, safe GC, orphan cleanup.
9 - Scale/HA	300-worker soak tests, S3 request tuning, failure injection, rolling upgrades.
10 - Production hardening	Rate limits, mTLS, auditing, backup/restore drills, quota and abuse controls.

26. Design limitations and future scale options

26.1 Hot top-level namespaces

The simplicity comes from never splitting a top-level namespace. If one organization becomes far larger or busier than others, its entire workload remains on one shard. That can create a hot shard even when overall repository distribution is balanced.

The base design intentionally accepts this trade-off. Mitigations that preserve the simple routing model include per-namespace quotas, stronger hardware for affected workers, shard owner pod vertical scaling, hot repository caches, and limiting exceptionally large top-level organizations. A future generation could add explicit split namespaces, but that would intentionally break the current no-routing-table property and should not be introduced prematurely.

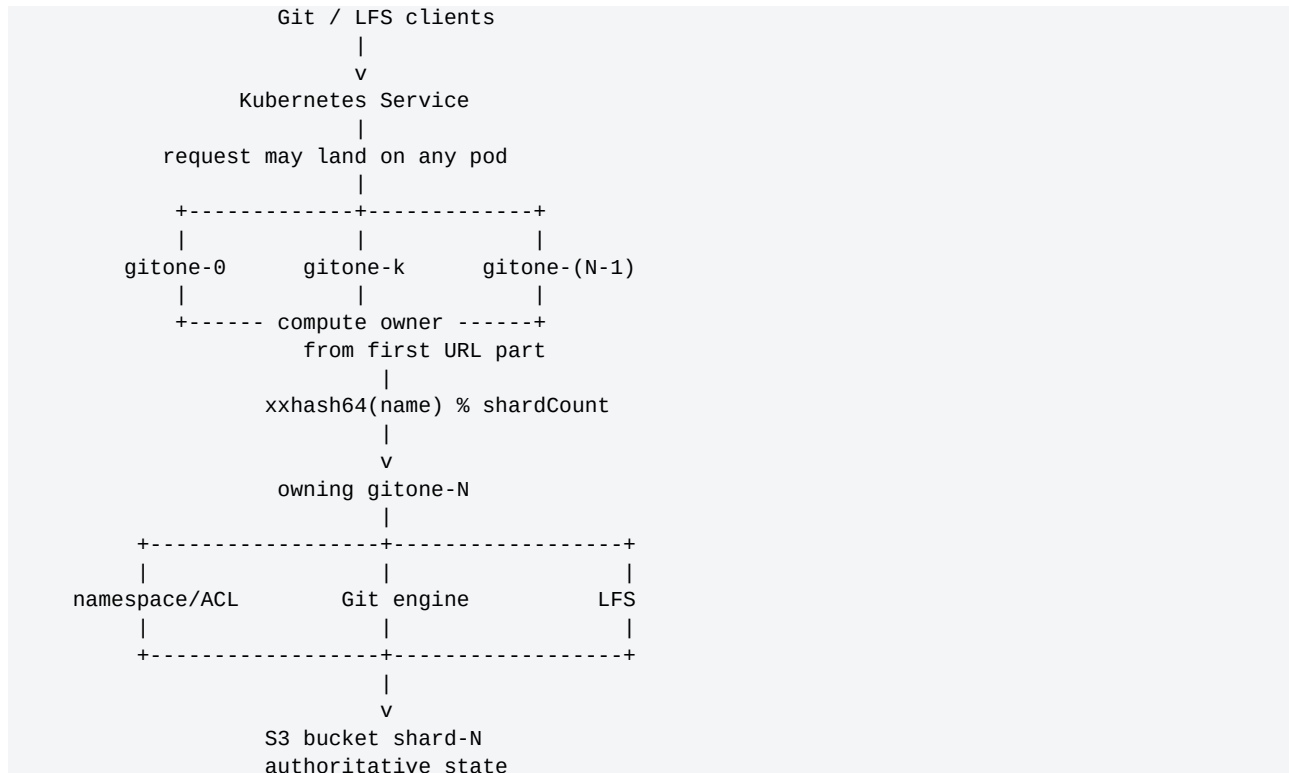
26.2 Shard count is configurable once

The shard count is an installation-time parameter. Routing uses $\text{xxhash64}(\text{topLevelNamespace}) \% \text{shardCount}$, so deployments may choose values such as 64, 128, 256, 512, or another positive count. Once the first namespace is created, `shardCount` becomes immutable for that installation: changing it would remap existing top-level names and require an explicit data-migration procedure. It is therefore a setup parameter, not an autoscaling setting.

26.3 Pod count versus compute scaling

The logical owner count is fixed to the shard count selected during installation. If one shard later needs more serving concurrency without changing ownership, it can be represented by multiple replicas behind a shard-local service with a single-writer/ref-transaction mechanism. That is an optional complexity and not part of the initial architecture.

27. Final architecture summary



Final decision	Value
Logical shards	N (configured at initial setup; 256 reference deployment)
Primary shard-serving pods	N - one per shard
Buckets	N, exactly one per shard

Routing key	Immutable first URL component / top-level namespace
Hash	xxhash64(top-level namespace) % shardCount
Separate router	No
Maximum forwarding	One internal hop
Top-level rename	Not supported
Login rename	Not supported
Subgroup/repository rename	Supported inside the same top-level namespace
S3 role	Only authoritative Git/LFS repository storage
Local disk	Ephemeral cache/workspace only
Git clients	Standard Smart HTTP clients
LFS	Supported and shard-local
Target scale	100M users, 2B project repositories, 300 Kubernetes workers

Architecture in one sentence

GitOne is a deterministic namespace-sharded Go Git service whose shard count is chosen once at installation; every pod can route any request, each immutable top-level namespace is permanently owned by one StatefulSet shard, and each shard persists all authoritative repository and LFS data in its own S3 bucket.

Appendix A - Naming and path rules

Item	Recommended rule
Login/top-level key	Lowercase ASCII letters, digits, hyphen and selected safe characters; immutable
Reserved names	Reject GitOne/system/API names
Case	Canonical lowercase; no case-distinct aliases
Path traversal	Reject ., .., empty canonical components and encoded separator tricks
Repository suffix	.git accepted/normalized by routing layer
Maximum component length	Policy-defined and enforced before hash/path processing
Maximum subgroup depth	Policy-defined to bound ACL/path processing

Appendix B - Suggested repository identifiers

Use immutable opaque repository IDs internally even though routing is path based. An ID may embed the shard byte as a debugging/operations convenience, for example AD-<random/ULID>. This is not used for external routing; it simply makes maintenance records self-describing.

```
repo ID: AD-01KXYZ...
AD hex = 173 decimal
bucket = gitone-prod-shard-173
```

Appendix C - Operational invariants

- The same canonical top-level name always hashes to the same shard.
- Top-level names and login names never change after allocation.
- Only owner shard N writes authoritative objects to bucket N.
- No published repository state references an object that was not durably uploaded first.
- Existing immutable packs are never edited in place.
- Every repository mutation has an old-state precondition.
- GC removes data only after proving it is unreachable from retained states.
- Forwarded requests are authenticated as pod-to-pod traffic and cannot loop.
- A pod can be recreated from S3 and configuration without restoring a local repository filesystem.